Distributed Leader–Follower Control Architecture for a Cooperative Drone Swarm in Simulation

Elena Ayala Ceballos
Degree in Robotic Intelligence
Universitat Jaume I
Castellón, Spain
al426665@uji.es

***Abstract*—This paper presents the design and validation of a cooperative multi-agent aerial system based on a distributed Leader–Follower architecture implemented in simulation. The system integrates multiple ArduPilot Software-In-The-Loop instances coordinated through ROS 2, where each drone operates as an independent agent with its own control script and MAVLink connection. The leader publishes position and velocity information, while followers compute local formation control actions using feedforward and proportional correction strategies. The architecture incorporates synchronization mechanisms and a coordinated landing strategy triggered by critical battery levels. Simulation results demonstrate coherent geometric formation maintenance and adaptive behavior under trajectory changes, validating the feasibility of distributed hierarchical coordination in aerial robotic swarms.**

***Index Terms*—Multi-agent systems, Drone swarm, Leader–Follower control, Distributed coordination, ArduPilot SITL, ROS 2, MAVLink**

I. Overview

This project presents the design and implementation of a cooperative multi-agent aerial system based on a Leader–Follower architecture, fully developed in simulation using ArduPilot SITL and coordinated through ROS 2. The objective is not merely simultaneous multi-drone flight, but the development of a distributed architecture capable of maintaining a coherent geometric formation during the motion of the leader.

The system consists of five autonomous agents: one leader and four followers. Each drone runs as an independent ArduCopter Software-In-The-Loop instance, establishing its own MAVLink connection and executing an individual Python control script. This separation ensures agent-level autonomy and avoids artificial centralized simulation, closely resembling a real multi-robot system.

Coordination is achieved through explicit state exchange. The leader periodically publishes its local position and velocity, while each follower computes its control action locally using only this shared information. Consequently, the formation is not determined by a centralized global controller but emerges from distributed control rules applied independently by each agent, following classical distributed coordination principles in multi-agent systems .

The leader implements a proportional position controller that generates velocity references in the local NED frame, while low-level stabilization is handled internally by the ArduPilot autopilot. Followers define their desired position relative to the leader's instantaneous motion direction, allowing the formation geometry to adapt dynamically to trajectory changes.

In addition to geometric control, the system incorporates a synchronization mechanism for coordinated formation activation and a battery monitoring strategy that enables a global landing event under critical conditions. Real-time visualization through RViz supports validation and analysis of collective behavior throughout the simulations.

II. Objectives

The main objective of this project is to design and validate a distributed multi-agent system capable of maintaining a dynamic aerial formation under a Leader–Follower architecture in simulation.

Specifically, the project aims to:

- Evaluate whether local control rules based solely on leader state exchange are sufficient to maintain coherent collective motion without a centralized global motion controller.
- Analyze the formation behavior resulting from independent agents executing identical geometric control strategies.
- Assess the feasibility of defining formation geometry with respect to the leader's velocity vector rather than a fixed global reference frame, enabling adaptive reconfiguration during trajectory changes.
- Empirically evaluate the effectiveness of simple proportional control strategies in maintaining bounded relative errors within a distributed cooperative system.
- Integrate event-based coordination mechanisms, including synchronized formation activation and coordinated landing triggered by critical battery levels.
- Validate system performance through real-time visualization tools, analyzing formation stability and dynamic transitions during maneuver execution.

III. System Architecture

The system architecture is organized into multiple functional layers that clearly separate physical simulation, individual control, and distributed coordination. Although the overall model

follows a Leader–Follower scheme, the structure is not based on a centralized motion controller. Instead, coordination relies on explicit state exchange through ROS 2, where topics act as formal communication channels between fully independent agents.

Each drone operates as an autonomous ROS 2 node maintaining its own MAVLink connection to a dedicated SITL instance. No direct function calls or internal dependencies exist between agents; coordination occurs exclusively through topic-based communication.

A. Simulation and Execution Layer

At the lowest level, independent ArduPilot Software-In-The-Loop (SITL) instances are executed. Each drone runs its own ArduCopter instance with a dedicated MAVLink port and differentiated initial GPS position.

The launch file orchestrates system initialization as follows:

- GPS positions are generated from metric offsets.
- SITL instances are started.
- ROS 2 nodes are launched with staggered delays.
- RViz is initialized once the system is fully operational.

This sequencing ensures that MAVLink connections are properly established before high-level communication begins.

B. Individual Control Layer

Each drone executes its own control script and operates as an autonomous agent. The leader implements a proportional navigation controller that generates velocity references, while the followers execute relative formation control.

At this level, agents do not possess global knowledge of other states, although the leader manages structural synchronization during initialization. Each node computes its behavior exclusively from subscribed topics. This separation reduces coupling and reinforces the distributed nature of the architecture.

C. Distributed Coordination Layer

Coordination is achieved entirely through ROS 2 state exchange.

1) Leader State Publication: After takeoff, the leader acts as the primary information source, periodically publishing:

- `/leader_local_position` (spatial reference),
- `/leader_velocity` (kinematic reference),
- `/leader_path` (historical trajectory for monitoring).

Separating position and velocity enables decoupling between kinematic estimation and geometric control.

2) Formation Synchronization: The synchronization mechanism relies on the topics:

- `/drone_ready`
- `/start_formation`

Each drone publishes its identifier after completing takeoff. The leader maintains an internal set of confirmed agents and publishes the start message only when all five nodes are ready.

This design makes the leader responsible for coordinated activation without acting as a centralized motion controller. Followers remain inactive until receiving the signal, activating the internal variable `formation_active`. This simultaneous transition guarantees structural coherence at the beginning of cooperative motion.

3) Formation Control Topology: Each follower subscribes to:

- `/leader_local_position`
- `/leader_velocity`

Relative reference computation is triggered upon velocity updates, introducing implicit synchronization based on the leader's publication frequency.

No direct communication exists between followers. The network topology is radial: the leader broadcasts state information, and followers compute local behavior independently. This structure reduces coupling and simplifies overall system design. The overall distributed communication structure is illustrated in Fig. 1.

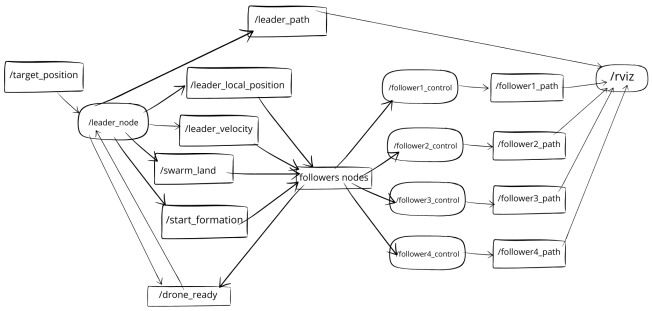


Fig. 1. Distributed Leader–Follower communication architecture. The leader publishes state information to follower control nodes, which compute local actions independently. Visualization data is forwarded to RViz.

4) Global Safety Channel: The system incorporates a distributed propagation mechanism for critical events with centralized detection at the leader via the `/swarm_land` topic.

When the leader detects battery level below a predefined threshold:

- A landing message is published.
- All followers receive the command.
- Each agent switches immediately to `LAND` mode via MAVLink.

Thus, an event detected by a single agent propagates structurally throughout the swarm without requiring an additional supervisory controller.

D. External Interaction Layer

The leader subscribes to the `/target_position` topic, enabling dynamic modification of navigation objectives. This demonstrates that the architecture is not a closed system, but capable of receiving high-level commands that alter collective swarm behavior.

E. Visualization and Spatial Consistency

Each node publishes its TF transform between the global `world` frame and its local `drone_name/base_link` frame.

TF consistency is essential for correct spatial representation in RViz, ensuring that all agents are visualized within a common reference frame. Additionally, publishing historical trajectories enables visual analysis of formation stability, dynamic transitions, and response to critical events.

F. Global Information Flow

From a structural perspective, information flow follows a radial scheme:

- The leader acts as the primary state broadcaster.
- Followers act as state consumers.
- Each agent generates its own local control action.
- No lateral communication exists between followers.

Overall, the architecture clearly separates simulation, local control, and distributed coordination through topic-based communication, maintaining low coupling and functional coherence.

IV. Technologies Used

The development of the cooperative swarm system relies on the coordinated integration of multiple specialized tools in aerial robotics, simulation, and distributed communication. Each technology fulfills a specific role within the overall swarm architecture.

A. ArduPilot SITL

The ArduPilot autopilot is used in Software-In-The-Loop (SITL) mode to simulate the behavior of an embedded flight controller. Each drone operates as an independent ArduCopter instance with its own MAVLink port and differentiated initial GPS configuration generated from metric offsets in the launch file.

The use of multiple SITL instances enables realistic multi-agent simulation, validation of distributed interactions, and avoidance of centralized simplifications. Additionally, ArduPilot provides internal attitude stabilization and low-level control loops, allowing the implemented system to operate exclusively at the level of velocity or position references without directly commanding actuators.

B. MAVLink and `pymavlink`

Communication with each SITL instance is performed through the MAVLink protocol using the `pymavlink` library. This interface enables:

- Flight mode transitions (GUIDED, LAND),
- Vehicle arming,
- Takeoff commands,
- Velocity references via `SET_POSITION_TARGET_LOCAL_NED`,
- Telemetry reception (local position, battery status, etc.).

In the leader, MAVLink is also used to monitor battery status through `BATTERY_STATUS` messages. When the battery level drops below a predefined threshold, a coordinated landing procedure is triggered.

C. ROS 2

ROS 2 acts as the distributed middleware enabling inter-agent communication through topics. In this project, it is used for:

- Publishing leader position and velocity,
- Distributed synchronization via `/drone_ready` and `/start_formation`,
- Broadcasting critical events through `/swarm_land`,
- Receiving dynamic navigation goals via `/target_position`,
- Publishing trajectories using `nav_msgs/Path`,
- Broadcasting TF transformations.

Using ROS 2 instead of direct socket-based communication ensures low coupling between nodes, scalability, modularity, and seamless integration with visualization tools.

D. RViz and TF

RViz is employed for real-time visualization of swarm behavior. Each drone publishes its transform between the global `world` frame and its local `base_link` frame, as well as its historical trajectory.

This enables visual verification of formation stability, dynamic transitions, directional changes, and coordinated landing events. TF consistency is essential for correctly representing the relative geometry of the swarm within a shared reference frame.

E. Python as Integration Language

All high-level control logic is implemented in Python, allowing rapid development, structural clarity, and seamless integration with both ROS 2 and MAVLink. Each drone executes its own independent control script (e.g., `leader.py`, `followerX.py`), reinforcing the distributed nature of the system.

V. Results

The system was evaluated entirely in simulation using multiple independent ArduPilot SITL instances coordinated through ROS 2. The experiments allowed analysis of both individual agent behavior and collective swarm dynamics under different operating conditions.

The results are structured according to the main system mechanisms: synchronization, leader control, formation maintenance, and critical event management.

A. Distributed Swarm Synchronization

Before cooperative motion begins, each drone publishes a readiness message on the `/drone_ready` topic. The leader waits until confirmation from all agents is received before publishing `/start_formation`, simultaneously activating formation control in the followers.

The following observations were made during simulation:

- No follower initiated movement before receiving the start signal.
- The transition to cooperative mode occurred synchronously.
- No premature displacements were observed during arming and takeoff.

This mechanism prevents inconsistent initial conditions and avoids geometric errors during the first seconds of flight.

An example of independent SITL instances during cooperative flight is shown in Fig. 2.

Fig. 2. Independent SITL instances showing spatial separation between leader and follower during cooperative flight.

B. Leader Proportional Control

The leader implements a proportional position controller in the horizontal plane to move toward a target defined in the local NED frame. The control law is expressed as:

$$v_x = K_p(x_{target} - x) \tag{1}$$

$$v_y = K_p(y_{target} - y) \tag{2}$$

Equations (1) and (2) define the velocity components as proportional functions of the position error, where (x, y) represents the current leader position, (x_{target}, y_{target}) the desired waypoint, and K_p the proportional gain.

Velocity commands are sent periodically using the `MAV_FRAME_LOCAL_NED` frame. A maximum velocity magnitude V_{MAX} is imposed to prevent excessive speeds.

1) Observed Dynamic Behavior: During simulation, the following dynamic phenomena were observed:

- **Exponential-like convergence toward the target:** The observed behavior is consistent with a first-order linear system operating in a non-saturated proportional regime. Velocity magnitude is maximal when the position error is large and decreases progressively as the leader approaches the target. This results in smooth trajectories without significant overshoot.
- **Automatic speed reduction near the target:** As the distance decreases, the magnitude of the commanded velocity vector is reduced proportionally, preventing abrupt approaches and improving motion smoothness.
- **Dead-zone via `STOP_RADIUS`:** When the distance to the target falls below a predefined threshold, the controller commands zero velocity. This eliminates micro-oscillations around the final position and improves steady-state stability.
- **Saturation regime:** For long displacements, the system initially operates in a saturated regime at constant maximum velocity. Once the error decreases sufficiently, the controller transitions to pure proportional behavior. This hybrid regime improves convergence time while maintaining final stability.
- **Robustness to small perturbations:** Small simulated disturbances or target modifications were automatically corrected by the proportional controller without introducing instability.

Since ArduPilot internally stabilizes attitude and velocity, the external controller operates on motion references rather than actuators. This decouples stabilization from navigation and reduces overall control complexity.

Overall, simulations showed:

- Stable behavior without sustained oscillations.
- Fast response to target changes.
- No accumulation of instability.
- Smooth trajectory transitions.

C. Geometric Formation Stability

Formation maintenance is based on a relative geometric reference defined by the instantaneous direction of leader motion rather than absolute position tracking. Such geometric formation approaches are widely studied in cooperative robotic systems.

Each follower computes its desired position as:

$$p_{desired} = p_{leader} - \hat{d}D_{back} + \hat{n}D_{side} \tag{3}$$

Equation (3) defines the relative geometric reference used for formation maintenance, where $\hat{d}$ is the unit vector aligned with the leader velocity, $\hat{n}$ its perpendicular unit vector, D_{back} the longitudinal offset, and D_{side} the lateral offset.

The geometric behavior of the formation during trajectory changes is illustrated in Fig. 3.

The commanded follower velocity is computed as:

$$v_{cmd} = v_{leader} + K(p_{desired} - p_{follower}) \tag{4}$$

Equation (4) combines feedforward and proportional correction. This control structure consists of two components:

Feedforward term (v_{leader}): Ensures the follower moves with the same base velocity as the leader.

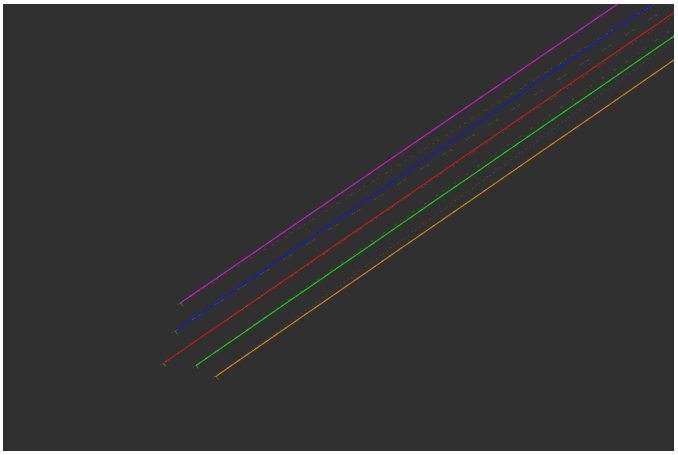


Fig. 3. RViz visualization of swarm formation during a trajectory change. The formation remains geometrically coherent while adapting to leader motion.

Proportional correction ($K \cdot error$): Reduces geometric deviation from the desired relative position.

1) Observed Formation Behavior: Equation (4) combines feedforward and proportional correction. The control structure consists of two components:

- **Feedforward term (v_{leader}):** Ensures that each follower moves with the same base velocity as the leader.
- **Proportional correction ($K \cdot error$):** Reduces geometric deviation from the desired relative position.

During simulation, the following formation behaviors were observed:

- **Straight-line motion stability:** When the leader moves at constant velocity in a straight line, the relative error decreases rapidly and followers reach stable positions without lateral oscillations. The formation remains rigid under these conditions.
- **Adaptation to trajectory changes:** When the leader changes direction, the unit direction vector $\hat{d}$ rotates dynamically, and each follower's desired position adapts accordingly. The formation reorients without redefining global offsets, demonstrating motion-based geometric definition.
- **Feedforward effect:** If only proportional control were applied,

$$v = K(p_{desired} - p_{follower}) \tag{5}$$

followers would exhibit significant delay during acceleration phases. Including the feedforward term,

$$v = v_{leader} + K(p_{desired} - p_{follower}) \tag{6}$$

substantially reduces transient error. Simulations showed that longitudinal spacing remains nearly constant and formation stretching during accelerations is minimal.
Comparing (5) and (6), the feedforward term reduces transient error during acceleration phases.

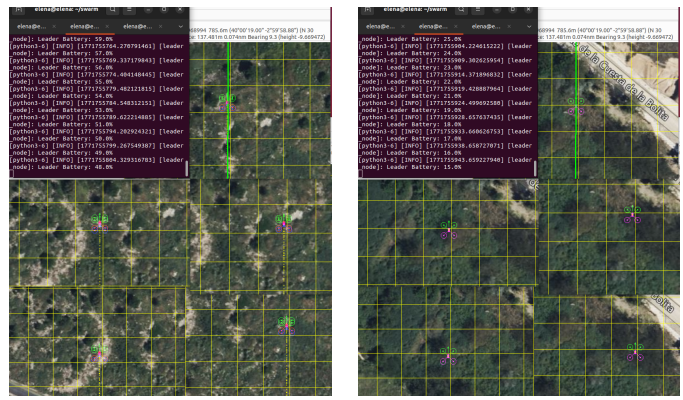
D. Battery Management and Coordinated Landing

The leader continuously monitors battery status through MAVLink `BATTERY_STATUS` messages. When the level falls below a predefined threshold:

- Formation is deactivated.
- The leader switches to LAND mode.
- The `/swarm_land` message is published.
- All followers immediately switch to LAND mode.

Simulations confirmed correct propagation of the landing command, synchronized transition, and absence of inconsistent behavior. This demonstrates that the system addresses fault scenarios beyond nominal operation.

The coordinated landing behavior is illustrated in Fig. 4.



(a) Nominal flight condition (b) After coordinated landing

Fig. 4. Coordinated landing triggered by a critical battery event.

E. Visualization and RViz Analysis

TF transforms and `nav_msgs/Path` messages enable real-time visualization of swarm behavior in RViz.

The following aspects were verified visually:

- Geometric coherence of the formation.
- Correct relative orientation of agents.
- Stability during extended maneuvers.
- Simultaneous transition during emergency landing.

Visualization proved essential for validating distributed coordination and understanding collective system behavior.

VI. Conclusions

The developed project demonstrates the technical feasibility of implementing a cooperative multi-agent aerial system based on a Leader–Follower architecture in simulation. Beyond simultaneous multi-drone execution, the results confirm that dynamic geometric formation can be maintained through distributed control and explicit state exchange.

A key aspect of the system is the separation between low-level stabilization and high-level mission control. Each drone delegates attitude and velocity stabilization to the ArduPilot autopilot, while cooperative logic is implemented externally in ROS 2. This layered architecture reflects real-world robotic platforms, where coordination operates above dynamic control loops.

The combined use of feedforward and proportional correction in follower control significantly reduces transient error during trajectory changes, improving overall formation stability. Because the formation is defined with respect to the leader's instantaneous velocity vector rather than a fixed global axis, the geometry adapts dynamically to changes in motion.

Furthermore, the distributed synchronization mechanism ensures simultaneous activation of cooperative behavior, preventing inconsistent initial conditions. The integration of battery monitoring and coordinated landing demonstrates that the system addresses emergency scenarios in addition to nominal operation.

A. Limitations and Future Work

Despite the positive results obtained in simulation, several limitations remain.

First, the implemented controller is purely proportional. While sufficient for stable behavior in simulation, real-world deployment would likely require more advanced strategies to compensate for wind disturbances, communication latency, and sensor noise. Future improvements could include derivative terms or robust control techniques.

Second, the Leader–Follower architecture introduces structural dependence on the leader. If the leader fails, the swarm loses its global reference. More resilient architectures, such as distributed consensus or graph-based coordination, could improve fault tolerance.

Additional enhancements may include inter-follower collision avoidance mechanisms and cooperative trajectory planning in obstacle-rich environments.

Finally, although validated using independent SITL instances, transitioning to real hardware would require further experimentation addressing communication interference, positioning accuracy, and timing constraints.

B. Ethical Considerations and Responsibility

The development of cooperative aerial robotic systems entails significant responsibility in terms of safety, privacy, and ethical deployment.

The implementation of coordinated landing under low battery conditions reflects a design decision prioritizing system and environmental safety. In real-world applications, such mechanisms are essential to prevent risks to people and infrastructure.

Moreover, validating the system in simulation before physical deployment represents a responsible engineering practice, reducing operational risk during development.

Drone swarms offer valuable societal applications, including environmental monitoring, search and rescue, infrastructure inspection, and precision agriculture. However, they may also raise concerns related to privacy and potential misuse. Responsible development therefore requires regulatory compliance and careful consideration of ethical implications.

SUPPLEMENTARY MATERIAL

Descriptive Video: A short video demonstrating project execution (maximum duration: 2 minutes) with explanatory audio is available at: https://drive.google.com/file/d/1ru0egv2hpoTmi7R75b4K2zdNPLECyNnZ/view?usp=sharing

Source Code: The complete source code of the project, including leader and follower control scripts, launch files, and configuration parameters, is available at: https://drive.google.com/drive/folders/1ey-JlAZPe_8JGvJUCtQBNUMsXIoNSJF3?usp=sharing